Lab 6: Code and Branch Coverage Analysis

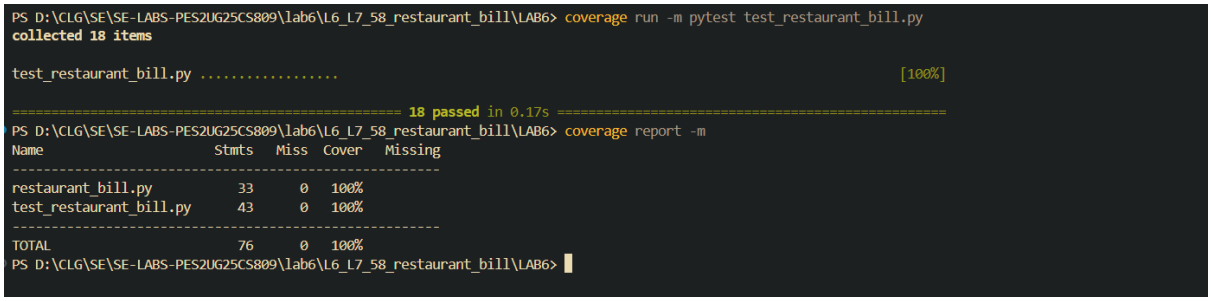
Name: Ibrahim Khaleel

SRN: PES2UG25CS809

Section: D

Screenshots





Code

```
import pytest

from restaurant_bill import bill_total, split_bill, table_wait


# ========================================

# bill_total Tests

# ========================================


def test_dine_in_bill():

    # Example test given

    assert bill_total("dine_in", 1000, 2, False) == 1155
```

```
def test_unknown_section():
    # Covers line 6: raise ValueError for unknown section
    with pytest.raises(ValueError, match="unknown section"):
        bill_total("patio", 1000, 2, False)


def test_invalid_food_total():
    # Covers line 8: raise ValueError for negative food total
    with pytest.raises(ValueError, match="invalid order"):
        bill_total("dine_in", -5, 2, False)


def test_invalid_guests():
    # Covers line 8: raise ValueError for 0 guests
    with pytest.raises(ValueError, match="invalid order"):
        bill_total("dine_in", 100, 0, False)


def test_happy_hour_dine_in():
    # Covers line 10: Happy hour discount applied (True) for non-takeaway
    # 1000 * 80//100 = 800; service = 80; total = 880; +5% = 924
    assert bill_total("dine_in", 1000, 2, True) == 924


def test_happy_hour_takeaway():
    # Covers line 10 False branch: Happy hour but takeaway (no discount)
    # takeaway service is 0; total 1000; +5% = 1050
    assert bill_total("takeaway", 1000, 2, True) == 1050


def test_large_party_extra_charge():
    # Covers line 13 True branch: guests >= 8
    # rooftop 15% of 1000 = 150; +200 for large party = 350; total 1350; +5% = 1417
    assert bill_total("rooftop", 1000, 10, False) == 1417
```

```
def test_small_party_no_extra_charge():
    # Covers line 13 False branch: guests < 8
    # rooftop 15% of 1000 = 150; total 1150; +5% = 1207
    assert bill_total("rooftop", 1000, 5, False) == 1207



# ========================================
# split_bill Tests
# ========================================

def test_even_split():
    # Example test given
    assert split_bill(1000, 4, 10) == 275

def test_invalid_people_count():
    # Covers line 20: raise ValueError
    with pytest.raises(ValueError, match="invalid people count"):
        split_bill(1000, 0, 10)

def test_invalid_tip_percent():
    # Covers line 22: raise ValueError
    with pytest.raises(ValueError, match="invalid tip"):
        split_bill(1000, 4, -1)

def test_uneven_split_rounds_up():
    # Covers line 26 True branch: grand_total % people != 0
    # 1000 + 10% tip = 1100. 1100 // 3 = 366. remainder > 0, so adds 1 = 367.
    assert split_bill(1000, 3, 10) == 367

def test_even_split_exact_division():
```

```
    # Covers line 26 False branch: grand_total % people == 0
    # 1100 / 4 = 275 exactly
    assert split_bill(1000, 4, 10) == 275


# ========================================
# table_wait Tests
# ========================================

def test_large_party():
    # Covers line 31: party_size > 12
    assert table_wait(13, 5, True) == "call ahead"

def test_has_reservation():
    # Covers line 33: has_reservation is True
    assert table_wait(4, 0, True) == "seat now"

def test_no_free_tables():
    # Covers line 35: free_tables == 0
    assert table_wait(4, 0, False) == "waitlist"

def test_large_party_few_tables():
    # Covers line 37 True branch: party_size > 6 and free_tables < 2
    assert table_wait(7, 1, False) == "waitlist"

def test_seat_now_fall_through():
    # Covers final line: none of the earlier conditions are met
    # party_size <= 6, no reservation, free_tables > 0
    assert table_wait(4, 3, False) == "seat now"
```